HW3 — Program analysis: bounded model checking, loop invariants, abstract interpretation

CS 3892 / CS 5892 · Formal Methods: Safe and Trustworthy AI · Fall 2026

Out	Monday, October 26
Due	**Tuesday, November 3, 11:59 p.m. US Central**
Weight	4% of the course grade (100 points)
Tools	CBMC 6 (bounded model checking of C, loop and function contracts, test generation), Z3, and the course's small abstract interpreter
Zero-install path	the HW3 notebook in Google Colab, or the FMAIV Codespace
Submit	Brightspace → Assignments → *HW3 — Program analysis (CBMC; abstract interpretation)*

Nothing to install. Every part of this runs in a browser tab.

Start here: github.com/ttj/cs3892-examples has the starters in `homework/hw3-program-analysis/`, and the HW3 notebook runs every one of them in Colab as it ships. Each starter runs as it is, with a placeholder where your work goes.

What you need, and when you get it. Tuesday Oct 27 (a C program as a transition system; CBMC; reading a trace; unwinding assertions) covers **Part 1**. Thursday Oct 29 (test generation with `cbmc --cover`; Hoare triples and the loop rule; CBMC loop contracts; abstract interpretation with signs and intervals, widening and the join; vacuous specifications) covers **Parts 2–5**. Every command you need is in this handout, so you can start any part early; Part 1 needs only Tuesday's class.

What this is for

CBMC answers questions about runs up to a bound. This homework is about knowing exactly what each answer covers, and about the two classic ways past the bound. **A loop invariant** proves a loop for every number of iterations, if you can find one that is established, preserved and sufficient. **Abstract interpretation** finds a coarser invariant by itself, always terminates, and pays for both with false alarms. And because the specification is where things go wrong — D7's question — the last part has an LLM write specifications and asks you to catch the ones that say nothing.

Running the tools

In Colab. Open the HW3 notebook and run it top to bottom. It installs Z3 and fetches CBMC. To make your changes, edit the starter files (the folder icon, left) or the cells, and re-run.

In the FMAIV Codespace, or any Linux shell:

```
git clone --depth 1 https://github.com/ttj/cs3892-examples.git
cd cs3892-examples/homework/hw3-program-analysis/python
pip install z3-solver
python3 cbmc_cli.py
export PATH="$HOME/.cache/cs3892/cbmc-6.11.0:$PATH"
cd ../c
cbmc counter.c counter_check.c --unwind 26
```

`cbmc_cli.py` fetches the official CBMC 6.11.0 release once, checks it, and puts `cbmc`, `goto-cc`, `goto-instrument` and `goto-analyzer` in `~/.cache/cs3892/cbmc-6.11.0/`. (The Codespace already has CBMC 6.9; either works.) On macOS, `brew install cbmc`.

Since CBMC 6, unwinding assertions are on by default. `--unwinding-assertions` is accepted and changes nothing; `--no-unwinding-assertions` turns them off. Tutorials written for CBMC 5 pass the first flag explicitly. All commands below are for CBMC 6.

Part 1 — CBMC: what a verdict at bound k covers *(20 points)*

`c/counter.c` is the course's two-mode counter (modes off and on, a button `press` input, a count that resets at 10). `c/counter_check.c` runs it for 25 steps from the initial state, with `press` free at every step, and asserts `s.x <= 10` after each one.

1. **Verify it.** Run `cbmc counter.c counter_check.c --unwind 26`. Report the verdict, and say why the bound is 26 for a 25-step harness. *(2)*
2. **Break it.** Copy the harness and weaken the assertion to `s.x < 10`. Run it at `--unwind 26` with `--trace`. From the trace, list the `press` inputs and the states in order, and say at which step `x` first reaches 10. CBMC's trace need not be the shortest one: **what is the fewest number of steps in which `x` can reach 10, and why can it not be fewer?** *(5)*
3. **Bound hunt.** Keep the weakened assertion, and turn unwinding assertions off with `--no-unwinding-assertions`. Find the smallest `--unwind` at which CBMC reports the failure, and explain that number using your answer to 2. *(5)*
4. **What the unwinding assertion is for.** Go back to the original, correct assertion. Run it at `--unwind 12` twice: once as is, once with `--no-unwinding-assertions`. Report both verdicts. Then explain, in two or three sentences of your own, what an unwinding assertion checks and what it protects you from. Use Part 1.3 as your example: which answer would you have trusted? *(4)*
5. **What `SUCCESSFUL` at bound 26 proves.** Does it prove that `x <= 10` on every run of the counter, of any length? If not, name two ways from this course to prove it for every length, and say in one sentence why each works. *(4)*

Part 2 — Loop invariants *(25 points)*

The loop rule (session 18): an invariant *Inv* proves {*Pre*} loop {*Post*} when it is **established** (*Pre* and the code before the loop give *Inv*), **preserved** (*Inv* and the loop condition, then one iteration, give *Inv*), and **sufficient** (*Inv* and the loop condition false give *Post*). An invariant can be true at every iteration and still fail: preserved can fail for a state no run reaches, and sufficient can fail because *Inv* says too little.

2a. `array_max`, in Z3

`python/hw3_part2_vcs.py` encodes `c/array_max.c` for `n >= 1`. Its postcondition: every `a[k]` with `0 <= k < n` is at most `m`, and `m` equals one of them. Each candidate in `CANDIDATES` gets the three conditions checked by Z3, with a counterexample when one fails. The encoding also tracks a **ghost variable** `w`, the index `m` was last copied from. It is not in the C code, and you may use it.

1. Add the obvious invariant — *"`m` is at least every element seen so far"* — in Z3 syntax (with whatever bounds on `i` it needs to be established). Run it. Which condition fails? Give Z3's counterexample and say in one sentence why the invariant is **true but not enough**. *(6)*
2. Find an invariant that passes all three. Explain what each conjunct is for, and say which condition fails if you remove it. Then give a **variant**, an expression that decreases on every iteration and is bounded below, and say what it proves that the three conditions do not. *(9)*

2b. `converge.c`, with a CBMC loop contract

`c/converge.c` starts `i` at 0 and `j` at 10 and moves them towards each other until `i < j` fails, then asserts `i == j`. `python/hw3_part2_converge.py "INV"` applies a CBMC loop contract with invariant `INV` and prints the three checks and, on a failure, the trace.

3. Run it with `"i <= j"`. That is true at every iteration of every run. Which check fails? Read CBMC's trace: what state does it start the failing iteration from, and can any run of the program reach that state? Explain why that is enough to fail. Then strengthen the invariant until CBMC reports `VERIFICATION SUCCESSFUL`, and explain why your invariant is inductive when `i <= j` is not. *(10)*

Part 3 — Abstract interpretation *(25 points)*

`python/absint.py` is the session 18 interpreter: signs, intervals, join and widening. `python/hw3_part3_programs.py` runs four programs, P1–P4, in the sign domain, in the interval domain, and in intervals with widening, and prints each variable's abstract value at the end.

1. **Tabulate.** For P1–P4, give each domain's result next to what the program actually computes. For every result that is less precise than the truth, say *which step* lost the precision: an arithmetic operation, a join, or widening. *(10)*
2. **P2's loop.** The loop runs exactly 10 times, yet plain join never reaches a fixpoint (the script stops it after 1000 rounds). Why? What does a relational domain know at the loop head that intervals do not, and what would it conclude about `total` at the end? *(5)*
3. **Your false alarm.** Write a program P5 — not P3, and not one from the slides — on which the interval domain raises a **false alarm because of a join**. Use an `assert(... != 0)`, as a guard before a division would. Add it to `PROGRAMS` in AST form, and write it in C in `c/part3_p5.c`. Show that `goto-analyzer part3_p5.c --verify --intervals` reports `UNKNOWN` and that `cbmc part3_p5.c` reports `SUCCESS`. Then say exactly what was lost at the join, and what constraint a relational domain would have kept. *(10)*

Part 4 — Generated tests: coverage is measurable, so measure it *(15 points)*

`c/triangle.c` classifies a triangle by its sides (0 not a triangle, 1 equilateral, 2 isosceles, 3 scalene). Compiled with `-DMUTANT`, it has one deliberate bug. `python/hw3_part4_coverage.py` takes test files (`a b c` per line), checks each result against the specification, measures branch coverage with `gcov`, and reports whether the suite catches the mutant.

1. **Generate.** Run `cbmc triangle.c --function triangle --cover branch --show-test-suite`. Put its inputs in `tests/cbmc.txt` and measure them. Report CBMC's coverage figure and `gcov`'s branch coverage, and explain why the two differ. *(5)*
2. **An LLM's tests.** Give an LLM `triangle.c` (without the `MUTANT` lines) and ask for unit tests. Put its inputs in `tests/llm.txt` and measure both suites together. Which suite catches the mutant? Explain how a suite with 100% branch coverage can miss it. *(7)*
3. In one or two sentences: what does branch coverage measure, and what does it not? *(3)*

Part 5 — AI component: contracts that say nothing *(15 points)*

`c/part5_functions.c` has three small functions: `sat_add` (saturating addition), `abs_diff` and `median3`. A CBMC **function contract** is a precondition (`__CPROVER_requires`) and a postcondition (`__CPROVER_ensures`, where `__CPROVER_return_value` is the result). The contracts live in `c/contracts.h`. `python/hw3_part5_check.py` checks each contract three ways:

- does the function satisfy it?
- does a **mutant** of the function (a version with a bug) also satisfy it? If so, the contract cannot tell the buggy version from the real one;
- can any input meet the precondition at all?

It also reports when CBMC finds an arithmetic overflow the precondition does not rule out.

Give an LLM the code and comments of the three functions (without the `MUTANT` lines) and ask it for contracts: CBMC syntax, or ACSL that you then translate. Put them in a copy of `contracts.h`, run the checker on it, and report:

- the prompt, and the LLM's contracts verbatim;
- the checker's table, and for each function what it shows: a contract the function fails, a contract the mutant also satisfies, a precondition nothing meets, an overflow left in;
- **vacuity**: any contract that is satisfied trivially — by the precondition alone, by every result, or because it only restates the code. Vacuity is the characteristic failure of written specifications, by people and by LLMs: look for it, and say plainly if you found none;
- your corrected contracts, which hold on the function, catch every mutant, and have meetable preconditions, with one sentence each on what you changed and why.

What to submit

One archive or repository link containing:

1. Your files, runnable as is: the Part 1 harness with the weakened assertion; your Part 2 `CANDIDATES` and your `converge.c` invariant; `PROGRAMS` with P5 and `c/part3_p5.c`; your `tests/cbmc.txt` and `tests/llm.txt`; the LLM's and your corrected `contracts.h`.

2. A short report — Markdown or PDF, roughly three pages — answering every numbered question, with the tool output it relies on.
3. The AI-use disclosure (see below).

The mechanical part is checked automatically: do your files run, and do they produce what you claim? The explanations are read by a human.

Late policy

Four late days for the semester, in whole-day units, no more than two on any one homework, no questions asked. Beyond that, 20% per day.

Collaboration and AI use

Discussion is encouraged; submitted work must be your own. **AI assistants are allowed and expected** — this is a course about the reliability of AI-produced artifacts. Two conditions:

1. **Disclose.** Say what you used and for what.
2. **Verify.** You are responsible for everything you submit, and "the model said so" does not transfer that responsibility. Every tool in this homework is a decision procedure that can tell you whether the model was right — so use one.